

ECS8055 ICW Part 1 — Dataset Preparation (GSE130051)

Objective: Create a clean and machine learning-ready dataset for the classification of 8 consensus Group 3/4 subtypes (I-VIII) based on DNA methylation.

Inputs (GEO):

- GSE130051_transposed_beta_values_GSE130051.csv.gz (beta value matrix)
- GSE130051-GPL13534_series_matrix.txt.gz (450K metadata)
- GSE130051-GPL21145_series_matrix.txt.gz (EPIC metadata)

Key requirement: Must retain platform information to be used in the analysis of robustness across platforms later.

Work in Part-1: View -> extract labels -> filter/clean -> feature feature selection -> produce final dataset.

Output: 'mb_part1_dataset.csv' with 'sample_id', 'platform', 'label', plus tens of thousands of CpG features.

```
import gzip
import re
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
```

1) Paths and file checks

- **Purpose:** Specify file paths and ensure all downloads are present.
- **Why it matters:** Avoids "file not found" and partial download problems early on.
- **Expected:** Transposed beta file is large, while series matrices are small.
- **If missing:** Copy files into the notebook directory or modify 'DATA_DIR'.
- **Next:** Series matrices are parsed to obtain sample IDs, platform, and subtype names.

```
DATA_DIR = Path(".")

SM_450K = DATA_DIR / "GSE130051-GPL13534_series_matrix.txt.gz"
SM_EPIC = DATA_DIR / "GSE130051-GPL21145_series_matrix.txt.gz"

BETA_COLS = DATA_DIR / "GSE130051_beta_values.csv.gz"
BETA_TRANS = DATA_DIR /
"GSE130051_transposed_beta_values_GSE130051.csv.gz"
```

```
for p in [SM_450K, SM_EPIC, BETA_COLS, BETA_TRANS]:
    print(p.name, "exists:", p.exists())
```

```
GSE130051-GPL13534_series_matrix.txt.gz exists: True
GSE130051-GPL21145_series_matrix.txt.gz exists: True
GSE130051_beta_values.csv.gz exists: True
GSE130051_transposed_beta_values_GSE130051.csv.gz exists: True
```

2) Parse Series Matrix metadata

- **Purpose:** Extract a table of samples ('sample_id') and their metadata.
- **Knowledge view:** Each sample is an entity with attributes: 'platform', 'characteristics'.
- **What we capture:** 'sample_id', 'platform', and 'characteristics_ch1' lines for label extraction.
- **Why it matters:** Subtype labels are stored inside 'characteristics_ch1'.
- **Output:** Combined metadata table across 450K + EPIC.

```
def parse_srs_matrix(srs_matrix_gz: Path, platform_name: str) ->
pd.DataFrame:
    fields = {}
    with gzip.open(srs_matrix_gz, "rt", encoding="utf-8",
errors="replace") as f:
        for line in f:
            line = line.rstrip("\n")
            if line.startswith("!srs_matrix_table_begin"):
                break
            if not line.startswith("!Sample_"):
                continue

            parts = line.split("\t")
            key = parts[0].replace("!Sample_", "")
            vals = parts[1:]

            if key == "characteristics_ch1":
                fields.setdefault(key, []).append(vals)
            else:
                fields[key] = vals

    if "geo_accession" not in fields:
        raise ValueError(f"Missing Sample_geo_accession in
{srs_matrix_gz.name}")

    sample_ids = [str(s).strip() for s in fields["geo_accession"]]
    df = pd.DataFrame({"sample_id": sample_ids})
    df["platform"] = platform_name

    chars_rows = fields.get("characteristics_ch1", [])
    chars_per_sample = [[] for _ in range(len(sample_ids))]
    for row in chars_rows:
        for i, v in enumerate(row):
```

```

        v = (v or "").strip()
        if v and v.upper() != "NA":
            chars_per_sample[i].append(v)

    df["characteristics"] = chars_per_sample
    return df

meta_450 = parse_srs_matrix(SM_450K, "450K")
meta_epi = parse_srs_matrix(SM_EPIC, "EPIC")
meta = pd.concat([meta_450, meta_epi], ignore_index=True)

meta["sample_id"] = meta["sample_id"].astype(str).str.replace("'", '',
regex=False).str.strip()
meta["characteristics"] = meta["characteristics"].apply(lambda lst:
[str(x).replace("'", '').strip() for x in lst])

print("Total samples in metadata:", len(meta))
print("Platform counts:\n", meta["platform"].value_counts())
meta.head()

Total samples in metadata: 1501
Platform counts:
platform
450K      1391
EPIC       110
Name: count, dtype: int64

   sample_id platform
characteristics
0  GSM3876338      450K [cohort: GSE93646, tissue: Medulloblastoma,
su...
1  GSM3876339      450K [cohort: GSE93646, tissue: Medulloblastoma,
su...
2  GSM3876340      450K [cohort: GSE93646, tissue: Medulloblastoma,
su...
3  GSM3876341      450K [cohort: GSE93646, tissue: Medulloblastoma,
su...
4  GSM3876342      450K [cohort: GSE93646, tissue: Medulloblastoma,
su...

```

3) Automatically discover the correct subtype key in GEO characteristics

- **Problem:** GEO may not call the field exactly 'subtype'.
- **Solution:** Search all 'characteristics_ch1' lines for text containing "subtype".
- **Why it matters:** If we choose the wrong key, label coverage becomes 0 and the dataset cannot be built.
- **Expected:** A list of characteristic lines showing where subtype is stored.
- **Next:** Use the discovered key to extract labels consistently.

```

QUOTE_CHAR = '""'\''

def normalize_text(s: str) -> str:
    s = str(s).strip()
    s = s.strip(QUOTE_CHAR)
    for ch in QUOTE_CHAR:
        s = s.replace(ch, "")
    return s.strip()

all_chars =
meta["characteristics"].explode().dropna().astype(str).map(normalize_t
ext)

subtype_lines = all_chars[all_chars.str.contains("subtype",
case=False, na=False)]
print("Found subtype-like characteristic lines:", len(subtype_lines))
print(subtype_lines.value_counts().head(30))

def _prefix(c: str) -> str:
    c = normalize_text(c)
    if ":" in c:
        return c.split(":", 1)[0].strip().lower()
    if "=" in c:
        return c.split("=", 1)[0].strip().lower()
    return ""

SUBTYPE_KEYS = sorted(set([_prefix(s) for s in subtype_lines.tolist()
if _prefix(s)]))
print("\nSubtype key candidates (prefixes):", SUBTYPE_KEYS)

if len(SUBTYPE_KEYS) == 0:
    raise ValueError(
        "No subtype keys found in Series Matrix characteristics."
        "Download MINiML or SOFT family files from GEO to extract
subtype labels.")

Found subtype-like characteristic lines: 1501
characteristics
subtype: 8      347
subtype: 7      303
subtype: 2      177
subtype: 4      140
subtype: MBNOS  131
subtype: 3      124
subtype: 6      120
subtype: 5      108
subtype: 1       51
Name: count, dtype: int64

Subtype key candidates (prefixes): ['subtype']

```

4) Extract and normalise subtype labels to I–VIII

- **Purpose:** Create the supervised learning target column 'label' (Subtype I–VIII).
- **Extraction:** Pull values from 'characteristics_ch1' using the discovered subtype key.
- **Normalisation:** Convert numeric 1–8 or text variants into Roman numerals I–VIII.
- **Quality check:** Report label coverage and subtype counts.
- **Output:** 'meta_labeled' which is samples with valid subtype labels.

```
def extract_from_characters(chars: list[str], keys: list[str]) -> str | None:
    keys_normalize = [normalize_text(k).lower() for k in keys]
    for c in chars:
        c2 = normalize_text(c)
        c_low = c2.lower()
        for k_low in keys_normalize:
            if c_low.startswith(k_low + ":") or c_low.startswith(k_low
+ "="):
                return c2.split(":", 1)[1].strip() if ":" in c2 else
c2.split("=", 1)[1].strip()
        return None

ROMAN = {"I":1,"II":2,"III":3,"IV":4,"V":5,"VI":6,"VII":7,"VIII":8}
IN_ROMAN = {v:k for k,v in ROMAN.items()}

def normalize_subtypes(x) -> str | None:
    if x is None:
        return None
    s = str(x).strip().upper().replace("SUBTYPE", "").strip()
    if re.fullmatch(r"[1-8]", s):
        return IN_ROMAN[int(s)]
    if s in ROMAN:
        return s
    m = re.search(r"\bVIII|VII|VI|IV|V|III|II|I\b", s)
    if m and m.group(0) in ROMAN:
        return m.group(0)
    return None

meta["subtype_raw"] = meta["characteristics"].apply(lambda ch:
extract_from_characters(ch, SUBTYPE_KEYS))
meta["label"] = meta["subtype_raw"].apply(normalize_subtypes)

print("Label coverage (%):", round(meta["label"].notna().mean()*100,
2))
print("Subtype counts:\n", meta["label"].value_counts(dropna=False))

meta_labeled = meta.dropna(subset=["label"]).copy()
print("meta_labeled rows:", len(meta_labeled))

meta_labeled[["sample_id", "platform",
```

```
"label"]].to_csv("mb_metadata_labeled.csv", index=False)
print("Saved -> mb_metadata_labeled.csv")
```

Label coverage (%): 91.27

Subtype counts:

label	count
VIII	347
VII	303
II	177
IV	140
None	131
III	124
VI	120
V	108
I	51

Name: count, dtype: int64

meta_labeled rows: 1370

Saved -> mb_metadata_labeled.csv

5) Match sample IDs: beta matrix columns

- **Problem:** Beta matrix columns are **chip IDs** (e.g., 'X6026818126_R01C01'), not GSM accessions.
- **Solution:** Establish correspondence between GEO metadata and chip IDs via 'characteristics'.
- **Alternative Solution:** Use **order** if chip IDs are not available in metadata.
- **Result:** We get 'wanted_cols' and corresponding GSM IDs for labels/platform.
- **Next:** Chunk-read the transposed beta matrix file to prepare 'X_ready'.

```
BETA_COLS = DATA_DIR / "GSE130051_beta_values.csv.gz"
```

```
BETA_TRANS = DATA_DIR /
"GSE130051_transposed_beta_values_GSE130051.csv.gz"
```

```
def read_header_field_gz(path, sep=","):
    with gzip.open(path, "rt", encoding="utf-8", errors="replace") as
f:
    line = f.readline().rstrip("\n")
    return [x.strip().strip('"') for x in line.split(sep)]
```

```
beta_cols = read_header_field_gz(BETA_TRANS)
print("Tall beta columns:", len(beta_cols), " first 5:",
beta_cols[:5])
```

```
def extract_chip_id(chars):
    joined = " | ".join(map(str, chars))
    m = re.search(r"(\d{7,})_R\d{2}C\d{2}", joined)
    if m:
        return "X" + m.group(0)
```

```

sentrrix_id = None
sentrrix_pos = None
for c in chars:
    s = str(c).strip()
    low = s.lower()
    if ("sentrrix" in low and "id" in low) or ("barcode" in low):
        m1 = re.search(r"\d{7,}", s)
        if m1:
            sentrrix_id = m1.group(0)
    if ("sentrrix" in low and "position" in low) or ("position" in
low):
        m2 = re.search(r"R\d{2}C\d{2}", s.upper())
        if m2:
            sentrrix_pos = m2.group(0)

    if sentrrix_id and sentrrix_pos:
        return f"X{sentrrix_id}_{sentrrix_pos}"
    return None

meta["chip_id"] = meta["characteristics"].apply(extract_chip_id)

print("Chip IDs extracted from metadata:",
meta["chip_id"].notna().sum(), "out of", len(meta))

beta_col_set = set(beta_cols)

chip_ok = meta["chip_id"].notna().mean() > 0.8 and
meta["chip_id"].isin(beta_col_set).mean() > 0.8

if chip_ok:
    col_to_gsm = dict(zip(meta["chip_id"], meta["sample_id"]))
    print(" Using chip_id-based mapping.")
else:
    if len(beta_cols) != len(meta):
        raise ValueError("Order-based mapping not possible: beta_cols
and meta lengths differ.")
    col_to_gsm = dict(zip(beta_cols, meta["sample_id"].tolist()))
    print(" Using order-based mapping (beta column order <-> GEO
sample order).")

print("Order mapping check (first 5):")
print("Mapped GSMs:", [col_to_gsm[c] for c in beta_cols[:5]])
print("Meta GSMs  :", meta["sample_id"].head().tolist())

labeled_set = set(meta_labeled["sample_id"].tolist())
wanted_cols = [c for c in beta_cols if col_to_gsm.get(c) in
labeled_set]
wanted_gsms = [col_to_gsm[c] for c in wanted_cols]

labeled_set = set(meta_labeled["sample_id"].tolist())

```

```

wanted_cols = [c for c in beta_cols if col_to_gsm.get(c) in
labeled_set]
wanted_gsms = [col_to_gsm[c] for c in wanted_cols]

print("Wanted columns:", len(wanted_cols))
print("Example mapping:", wanted_cols[0], "->", wanted_gsms[0] if
wanted_cols else "N/A")

if len(wanted_cols) == 0:
    raise ValueError("No labeled samples matched to tall beta columns.
Mapping failed: inspect chip_id extraction.")

Tall beta columns: 1501 first 5: ['X6026818126_R01C01',
'X6026818126_R04C01', 'X6026818136_R01C02', 'X6164655140_R01C01',
'X6164655140_R03C01']
Chip IDs extracted from metadata: 0 out of 1501
Using order-based mapping (beta column order <-> GEO sample order).
Order mapping check (first 5):
Mapped GSMs: ['GSM3876338', 'GSM3876339', 'GSM3876340', 'GSM3876341',
'GSM3876342']
Meta GSMs : ['GSM3876338', 'GSM3876339', 'GSM3876340', 'GSM3876341',
'GSM3876342']
Wanted columns: 1370
Example mapping: X6026818126_R01C01 -> GSM3876338

```

6) Load CpG probe IDs such as feature names and assign them to tall beta rows

- **Problem:** The tall beta file usually has no CpG column, meaning row names are not assigned.
- **Solution:** The probe ID list comes from the wide file header.
- **Efficiency:** We cache probe IDs to 'mb_probe_ids.npy' so we don't repeat this step.
- **Assumption:** The rows in the tall file are in the same CpG order as the wide header.
- **Outcome:** Each chunk of the tall beta file has the right CpG IDs as its index.

```

PROBE_NPY = DATA_DIR / "mb_probe_ids.npy"

def load_probe_id():
    if PROBE_NPY.exists():
        probes = np.load(PROBE_NPY, allow_pickle=True).tolist()
        print("Loaded cached probe IDs:", len(probes))
        return probes

    probes = read_header_field_gz(BETA_WIDE)
    probes = [p.strip().strip('"') for p in probes]

    print("Probe IDs loaded from wide header:", len(probes), "first
5:", probes[:5])

```

```

np.save(PROBE_NPY, np.array(probes, dtype=object))
print("Saved probe IDs cache ->", PROBE_NPY.name)
return probes

```

```
probe_ids = load_probe_id()
```

```
Loaded cached probe IDs: 453152
```

7) Chunked platform-aware filtering + top-variance feature selection

- **Data used:** Tall beta matrix (rows = CpGs, columns = samples as chip IDs).
- **Platform-aware QC:** Select CpGs with low missingness in both 450K and EPIC labeled subsets.
- **Feature selection:** Select top-N CpGs by variance to select “tens of thousands” of features.
- **Model readiness:** Median-impute missing values after feature selection.
- **Output:** 'X_ready' (GSM samples × CpG features) ready to export.

```

CHUNK_SIZE = 4000
MAX_MISSING_PLATFORM = 0.05
TOP_PROBES = 50000

meta_idx = meta_labeled.set_index("sample_id")
platforms = [meta_idx.loc[g, "platform"] for g in wanted_gsms]
mask450 = np.array([p == "450K" for p in platforms], dtype=bool)
masked = np.array([p == "EPIC" for p in platforms], dtype=bool)

print("Mask sizes 450K:", mask450.sum(), "EPIC:", masked.sum())

probe_kept = []
vars_kept = []

row_offset = 0
reader = pd.read_csv(BETA_TRANS, compression="gzip",
                    usecols=wanted_cols, chunksize=CHUNK_SIZE, engine="c")

for chunk in reader:
    chunk.index = probe_ids[row_offset:row_offset + len(chunk)]
    row_offset += len(chunk)

    M = chunk.to_numpy(dtype=np.float32)

    miss_450 = np.isnan(M[:, mask450]).mean(axis=1) if mask450.any()
    else np.zeros(len(chunk), dtype=np.float32)
    miss_ep = np.isnan(M[:, masked]).mean(axis=1) if masked.any()
    else np.zeros(len(chunk), dtype=np.float32)

```

```

    keep = (miss_450 <= MAX_MISSING_PLATFORM) & (miss_ep <=
MAX_MISSING_PLATFORM)
    if not np.any(keep):
        continue

    M_keep = M[keep, :]
    valid_count = np.sum(~np.isnan(M_keep), axis=1)
    ok = valid_count >= 2
    v = np.zeros(M_keep.shape[0], dtype=np.float32)
    if np.any(ok):
        v[ok] = np.nanvar(M_keep[ok, :], axis=1).astype(np.float32)

    probe_kept.append(chunk.index.to_numpy()[keep])
    vars_kept.append(np.nan_to_num(v, nan=0.0))

stats_probe = np.concatenate(probe_kept)
stats_vars   = np.concatenate(vars_kept)

print("Probes retained after shared-platform missingness filtering:",
len(stats_probe))

top_idx = np.argsort(-stats_vars)[:min(TOP_PROBES, len(stats_vars))]
top_probes = set(stats_probe[top_idx].tolist())
print("Top probes selected:", len(top_probes))

row_offset = 0
kept_chunk = []
reader2 = pd.read_csv(BETA_TRANS, compression="gzip",
usecols=wanted_cols, chunksize=CHUNK_SIZE, engine="c")

for chunk in reader2:
    chunk.index = probe_ids[row_offset:row_offset + len(chunk)]
    row_offset += len(chunk)

    sub = chunk.loc[chunk.index.intersection(top_probes)]
    if len(sub) > 0:
        kept_chunk.append(sub)

beta_small = pd.concat(kept_chunk, axis=0)
print("beta_small shape (probes x samples):", beta_small.shape)

beta_small.columns = wanted_gsms

X1 = beta_small.T
X1.index.name = "sample_id"
print("X shape (samples x probes):", X1.shape)

na_before = int(X1.isna().sum().sum())
X_ready = X1.fillna(X1.median(axis=0, skipna=True))
na_after = int(X_ready.isna().sum().sum())

```

```

print("Missing before:", na_before, "Missing after:", na_after)

values = X_ready.to_numpy(dtype=np.float32)
print("Beta range (min,max):", float(np.nanmin(values)),
      float(np.nanmax(values)))
print("Samples:", X_ready.shape[0])
print("Features CpGs:", X_ready.shape[1])
print("Subtype distribution:\n", meta_labeled["label"].value_counts())
print("Platform distribution:\n",
      meta_labeled["platform"].value_counts())
PCA_PROBES = 5000
use_cols = X_ready.columns[:PCA_PROBES] if X_ready.shape[1] >
PCA_PROBES else X_ready.columns

Z = PCA(n_components=2,
        random_state=0).fit_transform(X_ready[use_cols].to_numpy(np.float32))
plt.figure(figsize=(7,5))
plt.scatter(Z[:,0], Z[:,1], s=10)
plt.title("PCA (2D) on beta values (subset of probes)")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.show()

```

```

final_meta = meta_idx.loc[X_ready.index, ["platform", "label"]].copy()

```

Mask sizes 450K: 1271 EPIC: 99

Probes retained after shared-platform missingness filtering: 453152

Top probes selected: 50000

beta_small shape (probes x samples): (50000, 1370)

X shape (samples x probes): (1370, 50000)

Missing before: 0 Missing after: 0

Beta range (min,max): 4.709542736236472e-06 0.9996999502182007

Samples: 1370

Features CpGs: 50000

Subtype distribution:

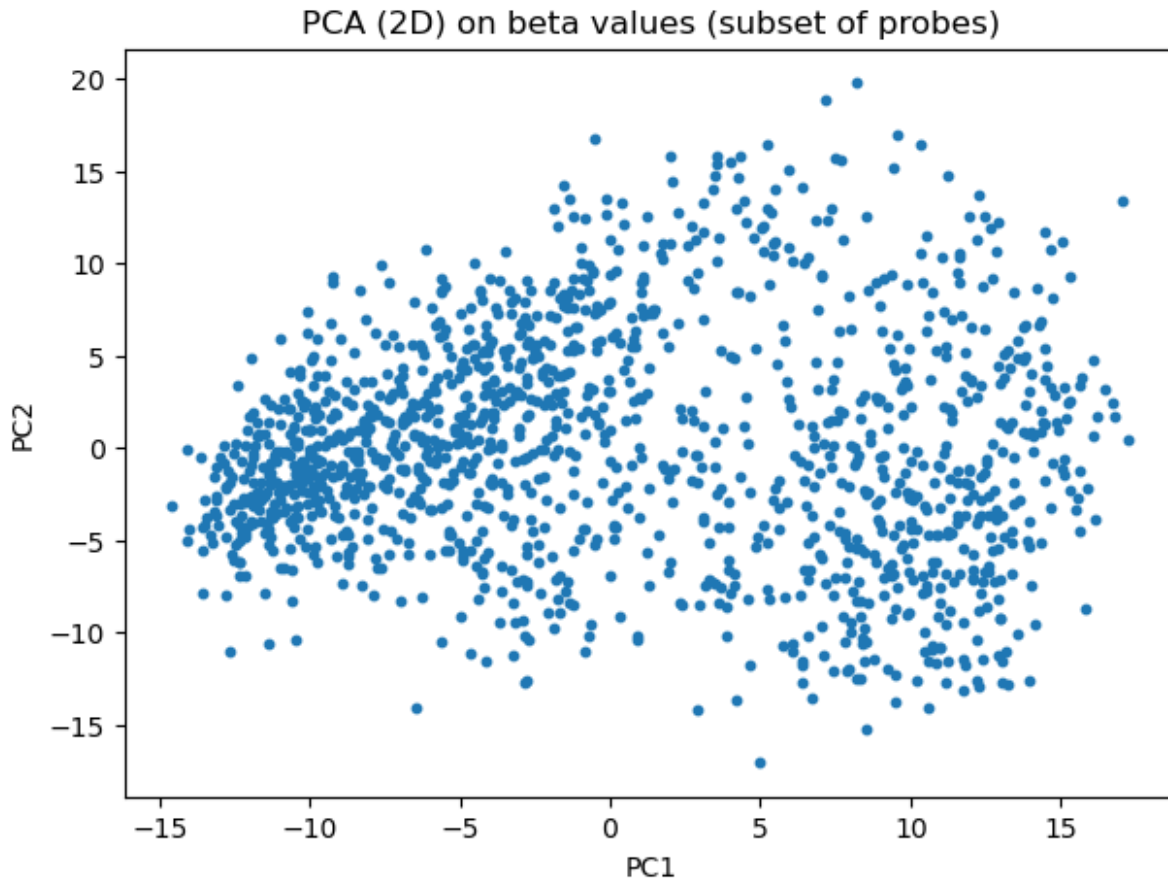
label	
VIII	347
VII	303
II	177
IV	140
III	124
VI	120
V	108
I	51

Name: count, dtype: int64

Platform distribution:

platform	
450K	1271
EPIC	99

Name: count, dtype: int64



8) Export final Part-1 dataset

- **Join:** Attach 'platform' and 'label' (I-VIII) to the CpG feature matrix using GSM 'sample_id'.
- **Deliverable:** One CSV ready for supervised classification.
- **Evidence:** Print counts and verify there are tens of thousands of CpG features.
- **Output file:** 'mb_part1_dataset.csv'.

```
final_meta = meta_idx.loc[X_ready.index, ["platform", "label"]].copy()
final = final_meta.join(X_ready, how="inner").reset_index()

cpg_cols = [c for c in final.columns if str(c).startswith("cg")]

print("Final dataset shape:", final.shape)
print("CpG feature columns:", len(cpg_cols))
print("\nPlatform counts:\n", final["platform"].value_counts())
print("\nSubtype counts:\n", final["label"].value_counts())

OUT_PATH = "mb_part1_dataset.csv"
final.to_csv(OUT_PATH, index=False)
print("\nSaved ->", OUT_PATH)
```

```
Final dataset shape: (1370, 50003)
CpG feature columns: 49940
```

```
Platform counts:
platform
450K      1271
EPIC        99
Name: count, dtype: int64
```

```
Subtype counts:
label
VIII      347
VII        303
II         177
IV         140
III        124
VI         120
V          108
I           51
Name: count, dtype: int64
```

```
Saved -> mb_part1_dataset.csv
```

9) Final dataset validation

- **Purpose:** Validate the dataset to ensure it has all the necessary columns and the expected scale of features.
- **Checks:** Check for the presence of 'platform' and 'label' columns and ensure they are completely filled.
- **Feature scale:** Check the number of CpG probe columns.
- **Sanity:** Document the final number of samples as proof of submission.
- **Outcome:** Once these steps are completed, the dataset will be ready.